STYLE HUB MENS FASHION E-COMMERCE PLATFORM

23CS54C – MODERN WEB TECHNOLOGIES

Submitted by

AKILANETHRAN 2403957610421121

In partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



NATIONAL ENGINEERING COLLEGE

(An Autonomous Institution affiliated to Anna University, Chennai)

K.R.NAGAR, KOVILPATTI - 628503

OCTOBER - 2026

NATIONAL ENGINEERING COLLEGE

(An Autonomous Institution affiliated to Anna University, Chennai)

K.R.NAGAR, KOVILPATTI - 628503

BONAFIDE CERTIFICATE

This is to certify that this project report, "Style Hub A Mens Fashion E-commerce Platform", is the bonafide work of **AKILANETHRAN (2403957610421121)** who carried out the project work under my supervision.

Course Instructor/Guide

Submitted to the **23CS54C – MODERN WEB TECHNOLOGIES** Viva-Voce examination held at National Engineering College, K.R.Nagar, Kovilpatti on _____

Internal Examiner **Co Examiner**

TABLE OF CONTENTS

Chapter	Title	Page
1	Introduction	4
2	Objectives	5
3	Description	6
4	Class Diagram	8
5	Table Structures	9
6	ER Diagram	16
7	Tech Stack and Architecture	17
8	Modules	18
9	Conclusion	47

CHAPTER 1

INTRODUCTION

Online fashion shopping gives customers access to many products, but choosing a suitable garment from pictures and product descriptions can still be difficult. A shopper must compare price, size, appearance, availability, and matching pieces while moving between multiple pages. A conventional catalogue provides the product record but does little to connect the selection, purchase, and personal styling decisions. StyleHub addresses this need through one men fashion web application that brings those activities together.

The system allows a registered user to browse a catalogue, inspect product details, view recently accessed items, save favourites, manage a cart, and purchase individual products, a coordinated outfit, or cart contents. It also adds social features: users can find friends, share selected wishlist products privately, and exchange reactions and comments. A rule based recommendation component suggests complementary garments for eligible products, while a deterministic chatbot answers common product and shopping questions. These two components use defined application logic rather than a trained conversational model.

For supported garments, StyleHub can send a customer photograph and a product garment image to a separate local FASHN VTON service and display the resulting virtual try-on image. This feature depends on the Python service, its model weights, and available hardware. The core shopping application continues to provide catalogue and purchase functions even when try-on is unavailable. Settings cover language, appearance, and notification preferences. An administrator has a restricted dashboard for product updates and operational summaries.

The application is implemented with a React and Vite frontend, a Node.js and Express REST API, and an SQLite database. JWT authentication identifies the user on protected requests, while the database records products, carts, purchases, social relationships, notifications, and try-on results. Optional provider integrations support Google sign-in and purchase communication when configured. This report documents the implemented modules, their data structures and APIs, and the interfaces to be illustrated by screenshots during final submission.

CHAPTER 2

OBJECTIVES

- To provide secure registration, login, profile management, and protected user and administrator pages.
- To offer a men fashion catalogue with product details, stock, prices, and recently accessed items.
- To support size aware cart management, applicable rewards, checkout, and purchase history.
- To let users save favourites and revisit products without repeating a catalogue search.
- To allow users to connect with friends and privately share selected wishlist products.
- To record reactions and comments on shared selections for collaborative style decisions.
- To generate rule based complementary outfit suggestions for eligible products.
- To answer common product and shopping questions through a deterministic assistant.
- To offer virtual try-on for supported garments through a local Python service when available.
- To maintain notification records and user communication preferences.
- To give administrators a restricted dashboard for metrics, users, purchases, and product updates.
- To support English, Tamil, and Hindi interface options with a saved language choice.
- To keep user purchases and social records separated by authenticated account.

CHAPTER 3

DESCRIPTION

StyleHub is a full stack men fashion application consisting of the following major functional areas. Each area is connected through the same user account and product catalogue.

User Authentication and Access Control

Registration and sign-in establish an account identity. Passwords are checked against stored hashes, and a JWT protects customer API requests. Profile editing and username completion support account maintenance, including Google-created accounts when that integration is configured. An ADMIN role controls access to the management dashboard.

Product Discovery and Personal Shopping

The catalogue displays product names, descriptions, images, categories, prices, stock, and applicable virtual try-on support. Product detail pages expose purchase, cart, wishlist, and outfit suggestion actions. Recently accessed records help a signed-in shopper return to previously viewed products. The cart retains selected products, sizes, and quantities before checkout.

Purchase and Communication

The purchase functions handle a single product, a coordinated combination, or all eligible cart items. They create purchase and purchase item records, calculate totals and applicable cart rewards, and update stock. A purchase history page lets the customer review saved transactions. In-app notifications and channel delivery records support purchase communication; external email, SMS, or WhatsApp delivery depends on configured providers.

Social Wishlist and Style Assistance

Wishlists store favourites separately from a cart. A user can send a friend request, accept a connection, share selected products, and respond to a shared list with feedback. Rule based outfit suggestions pair eligible products with complementary items. The chatbot answers defined styling and catalogue questions using application rules and product data.

Virtual Try-On and Administration

Virtual try-on accepts an uploaded customer image for a supported garment and requests local FASHN VTON inference. The generated result is saved for later viewing when the service succeeds. The administrator dashboard provides restricted access to metrics, catalogue updates, user summaries, purchase records, and try-on activity.

Technology and Preferences

React and Vite render the browser interface, Express exposes REST endpoints, and SQLite persists the application data. The interface supports English, Tamil, and Hindi and keeps the selected language. Appearance and communication settings are stored through the user settings flow. The optional Google sign-in and messaging integrations require separate provider credentials.

CHAPTER 4

CLASS DIAGRAM

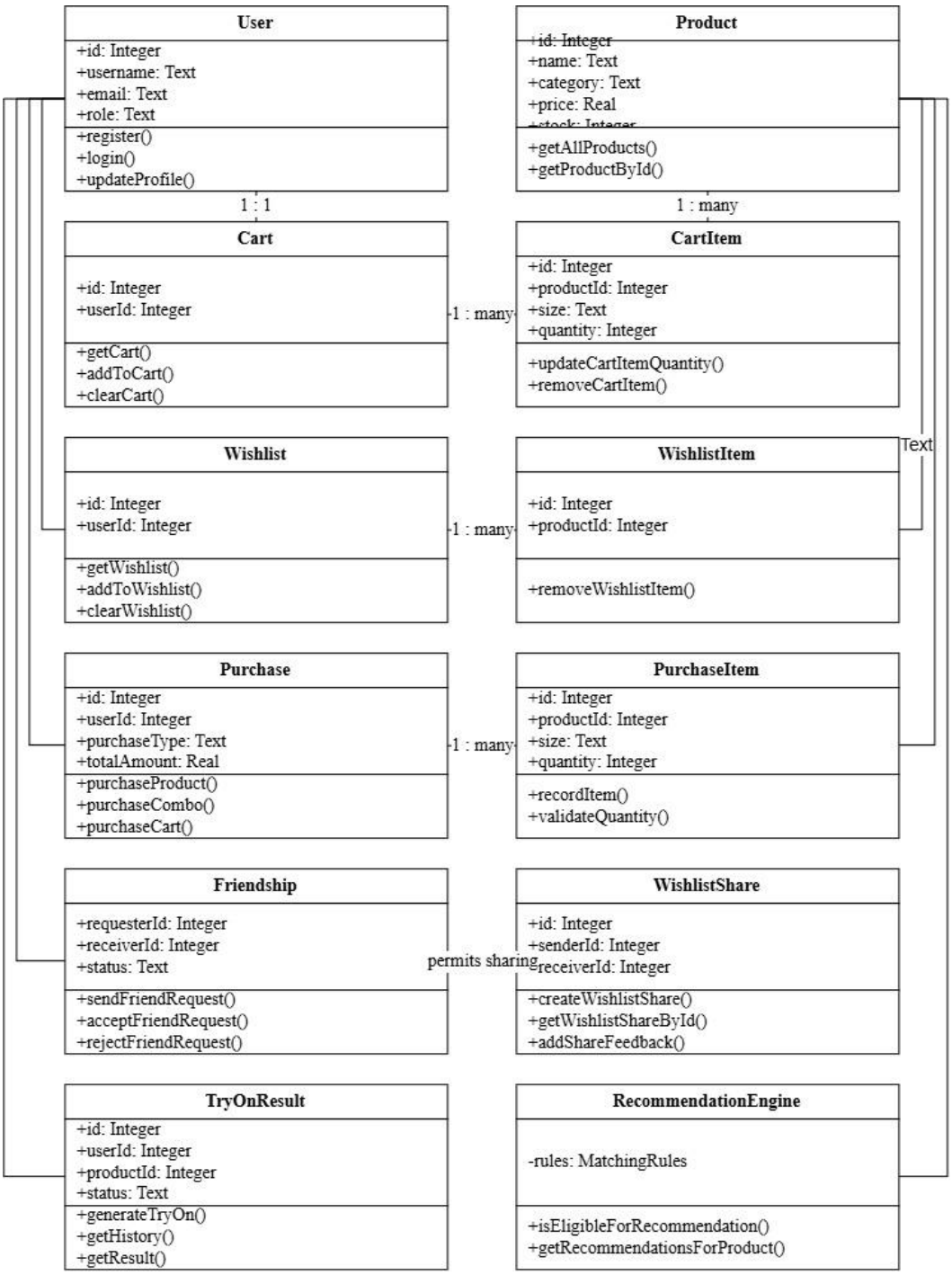


Fig 4.1 StyleHub Class Diagram

The class diagram shows the principal entities of StyleHub, including users, products, carts, purchases, wishlists and try-on results, together with their attributes and the relationships between them.

CHAPTER 5

TABLE STRUCTURES

The following entries are taken from the SQLite schema in the project. PK denotes primary key and FK denotes foreign key. The tables store both shopping records and optional feature records such as social sharing and virtual try-on.

Cart

Field	Type	Key	Required
id	INTEGER	PK	No
user_id	INTEGER	FK → users	Yes
created_at	DATETIME		No
updated_at	DATETIME		No

Cart Items

Field	Type	Key	Required
id	INTEGER	PK	No
cart_id	INTEGER	FK → cart	Yes
product_id	INTEGER	FK → products	Yes
size	TEXT		No
quantity	INTEGER		Yes
created_at	DATETIME		No
updated_at	DATETIME		No

Friendships

Field	Type	Key	Required
id	INTEGER	PK	No
requester_id	INTEGER	FK → users	Yes
receiver_id	INTEGER	FK → users	Yes
status	TEXT		Yes
created_at	DATETIME		No
updated_at	DATETIME		No

Notification Deliveries

Field	Type	Key	Required
id	INTEGER	PK	No
purchase_id	INTEGER	FK → purchases	Yes
user_id	INTEGER	FK → users	Yes
channel	TEXT		Yes
status	TEXT		Yes
recipient	TEXT		No
provider_message_id	TEXT		No
error_message	TEXT		No
event_type	TEXT		No
subject	TEXT		No
created_at	DATETIME		No

Notifications

Field	Type	Key	Required
id	INTEGER	PK	No
user_id	INTEGER	FK → users	Yes
type	TEXT		Yes
title	TEXT		Yes
message	TEXT		Yes
is_read	INTEGER		Yes
created_at	DATETIME		No
related_id	INTEGER		No

Order Items

Field	Type	Key	Required
id	INTEGER	PK	No
order_id	INTEGER	FK → orders	Yes
product_id	INTEGER	FK → products	Yes
product_name	TEXT		Yes
product_image	TEXT		No
quantity	INTEGER		Yes
unit_price	REAL		Yes
discount	REAL		Yes
final_price	REAL		Yes
size	TEXT		No

Orders

Field	Type	Key	Required
id	INTEGER	PK	No
order_number	TEXT		Yes
user_id	INTEGER	FK → users	Yes
status	TEXT		Yes
subtotal	REAL		Yes
discount_total	REAL		Yes
shipping_fee	REAL		Yes
other_charges	REAL		Yes
total_amount	REAL		Yes
delivery_address	TEXT		Yes
created_at	DATETIME		No
updated_at	DATETIME		No
delivery_date	DATETIME		No
cancelled_at	DATETIME		No

Outfit Recommendations

Field	Type	Key	Required
id	INTEGER	PK	No
base_product_id	INTEGER	FK → products	Yes
recommended_product_id	INTEGER	FK → products	Yes
slot_type	TEXT		Yes
reason	TEXT		No
created_at	DATETIME		No
combination_signature	TEXT		No

Products

Field	Type	Key	Required
id	INTEGER	PK	No
name	TEXT		Yes
description	TEXT		No
brand	TEXT		No
category	TEXT		Yes
cloth_type	TEXT		No
color	TEXT		No
price	REAL		Yes
original_price	REAL		Yes
discount_percent	REAL		No
stock	INTEGER		Yes
status	TEXT		Yes
image	TEXT		No
garment_image	TEXT		No
vton_supported	INTEGER		No
vton_garment_category	TEXT		No
created_at	DATETIME		No
updated_at	DATETIME		No

Purchase Items

Field	Type	Key	Required
id	INTEGER	PK	No
purchase_id	INTEGER	FK → purchases	Yes
product_id	INTEGER	FK → products	Yes
quantity	INTEGER		Yes
unit_price	REAL		Yes
created_at	DATETIME		No
size	TEXT		No

Purchases

Field	Type	Key	Required
id	INTEGER	PK	No
user_id	INTEGER	FK → users	Yes
purchase_type	TEXT		Yes
total_amount	REAL		Yes
delivery_name	TEXT		Yes
delivery_phone	TEXT		Yes
delivery_address	TEXT		Yes
delivery_city	TEXT		Yes
delivery_state	TEXT		Yes
delivery_pincode	TEXT		Yes
created_at	DATETIME		No
cart_reward_percent	REAL		No
cart_reward_amount	REAL		No
subtotal	REAL		No

Recently Accessed

Field	Type	Key	Required
id	INTEGER	PK	No
user_id	INTEGER	FK → users	Yes
product_id	INTEGER	FK → products	Yes
accessed_at	DATETIME		No

Try On Results

Field	Type	Key	Required
id	INTEGER	PK	No
user_id	INTEGER	FK → users	Yes
product_id	INTEGER	FK → products	Yes
user_image_path	TEXT		Yes
garment_image_path	TEXT		Yes
generated_image_path	TEXT		No
category	TEXT		Yes
status	TEXT		Yes
created_at	DATETIME		No

Users

Field	Type	Key	Required
id	INTEGER	PK	No
username	TEXT		No
full_name	TEXT		Yes
email	TEXT		Yes
password_hash	TEXT		Yes
role	TEXT		Yes
phone	TEXT		No
age	INTEGER		No
address	TEXT		No
city	TEXT		No
state	TEXT		No
pincode	TEXT		No
created_at	DATETIME		No
updated_at	DATETIME		No
google_id	TEXT		No
avatar_url	TEXT		No
email_notifications	INTEGER		Yes
sms_notifications	INTEGER		Yes
whatsapp_notifications	INTEGER		Yes
theme_preference	TEXT		Yes
language_preference	TEXT		Yes

Wishlist Feedback

Field	Type	Key	Required
id	INTEGER	PK	No
share_id	INTEGER	FK → wishlist_shares	Yes
user_id	INTEGER	FK → users	Yes
reaction	TEXT		No
comment	TEXT		No
created_at	DATETIME		No
updated_at	DATETIME		No

Wishlist Items

Field	Type	Key	Required
id	INTEGER	PK	No
wishlist_id	INTEGER	FK → wishlists	Yes
product_id	INTEGER	FK → products	Yes
created_at	DATETIME		No

Wishlist Share Items

Field	Type	Key	Required
id	INTEGER	PK	No
share_id	INTEGER	FK → wishlist_shares	Yes
product_id	INTEGER	FK → products	Yes
created_at	DATETIME		No

Wishlist Shares

Field	Type	Key	Required
id	INTEGER	PK	No
sender_id	INTEGER	FK → users	Yes
receiver_id	INTEGER	FK → users	Yes
wishlist_id	INTEGER	FK → wishlists	No
share_type	TEXT		No
created_at	DATETIME		No

Wishlists

Field	Type	Key	Required
id	INTEGER	PK	No
user_id	INTEGER	FK → users	Yes
created_at	DATETIME		No
updated_at	DATETIME		No

CHAPTER 6

ER DIAGRAM

The accompanying editable Draw.io ER diagram summarizes the main foreign key relationships. It includes shopping, social, recommendation, notification, recently accessed, and try-on data. One user can have many purchases; each purchase has many purchase items. Product references link items and feature records back to the catalogue.

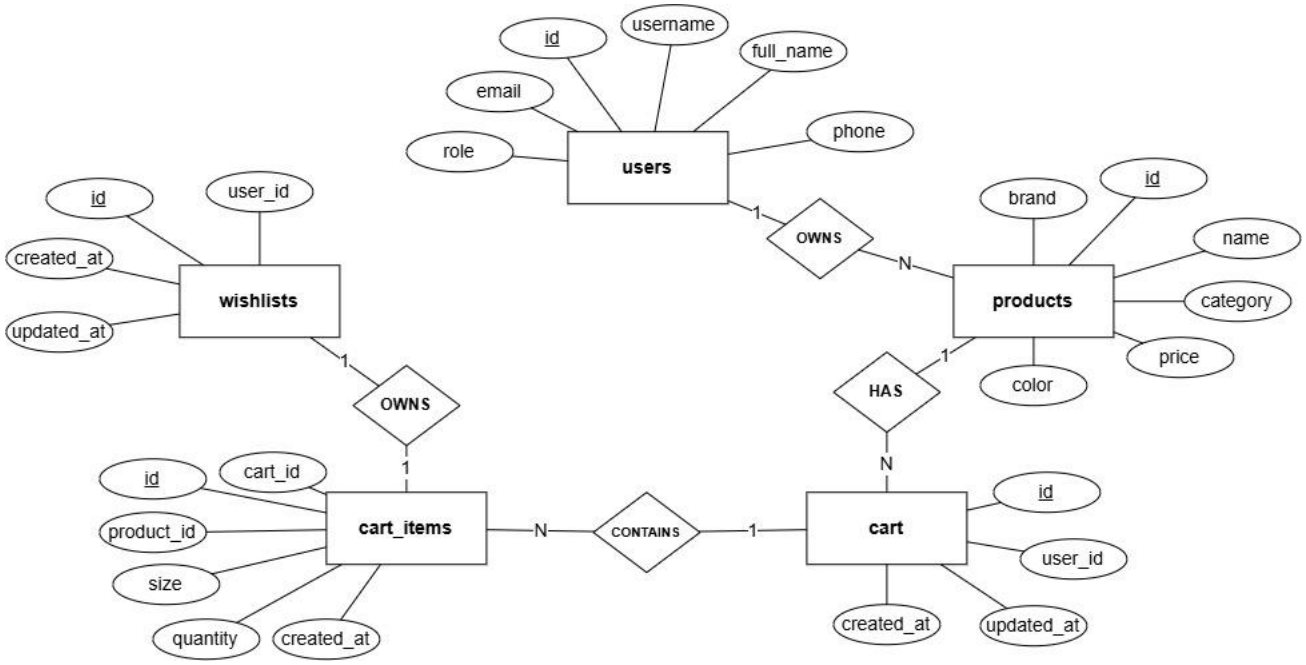


Fig 6.1 StyleHub Entity Relationship Diagram

The ER diagram shows how users, products, carts, purchases, wishlists, outfit recommendations, notifications, recently accessed records and try-on results are linked through foreign keys.

CHAPTER 7

TECH STACK AND ARCHITECTURE

Technology Stack

Layer	Technology	Purpose
Frontend	React 18, Vite 6, React Router	Screens, navigation, user state
Backend	Node.js, Express 4	REST API and business rules
Database	SQLite, better-sqlite3	Persistent records and transactions
Authentication	JWT, bcryptjs	Tokens and password hashing
Virtual try-on	Python Flask, FASHN VTON 1.5	Local image inference for supported garments
Communication	Nodemailer, Twilio SMS, Meta WhatsApp API	Optional configured purchase notifications

Architecture Overview

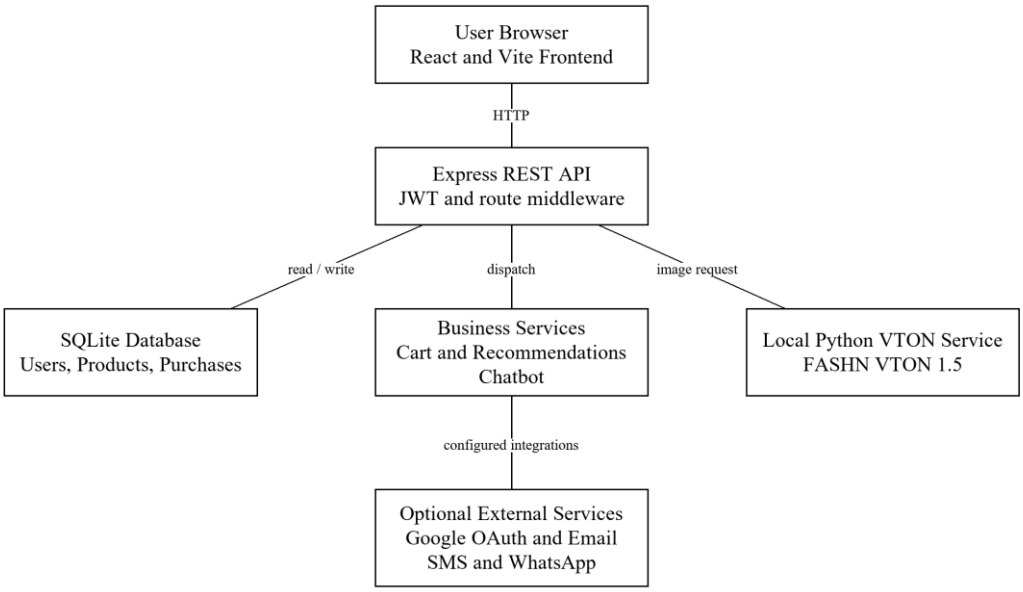


Fig 7.1 StyleHub System Architecture

The architecture diagram shows the React frontend, the Express REST API, the SQLite database, and the optional local VTON service and communication providers, along with the direction of requests between them.

CHAPTER 8

MODULES

8.1 Folder Structure

stylehub/stylehub-frontend contains React pages, reusable components, contexts, and frontend service calls. stylehub/stylehub-backend contains Express routes, controllers, services, middleware, database configuration, and tests. The VTON service is housed under the backend vton-service directory.



Fig 8.1 Overall Project Folder

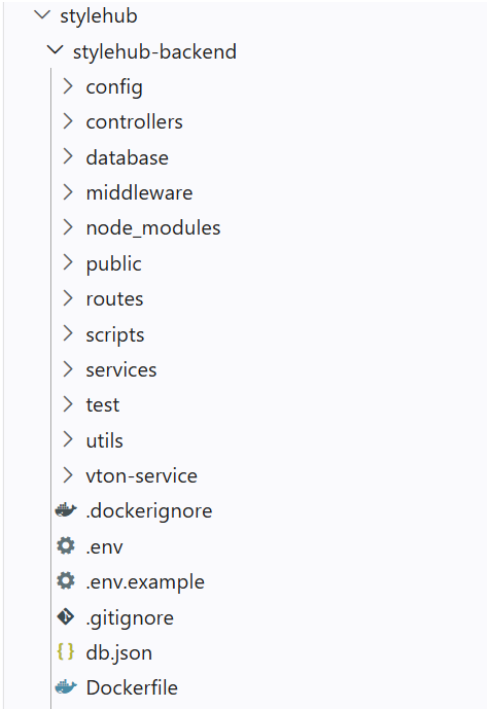


Fig 8.2 Backend Folder Structure

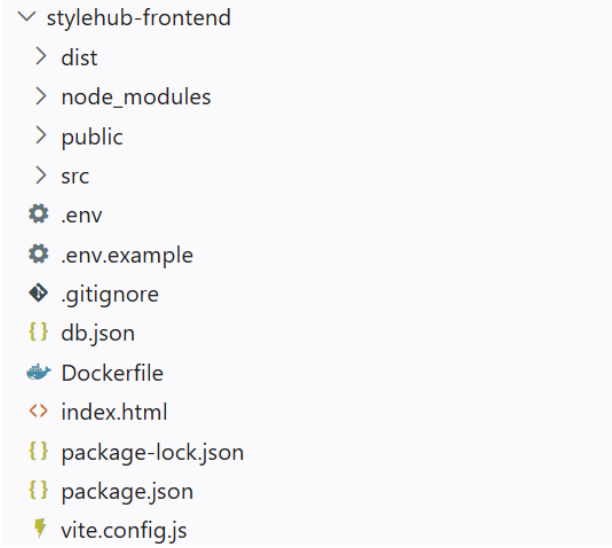


Fig 8.3 Frontend Folder Structure

8.2 Authentication and Profile

Purpose and Scope

This module creates and maintains customer identities, protects account data and distinguishes an administrator from a customer. It covers registration, credential login, profile updates, username availability and the optional Google sign-in path.

Functional Requirements

- Register a user with required identity and credential details.
- Authenticate a returning user and return a signed session token.
- Load and update only the signed-in user profile.
- Allow a Google-created account to complete a unique username.
- Restrict administrator pages and APIs to accounts with the ADMIN role.

Non Functional Requirements

- Store password hashes rather than plaintext passwords.
- Reject missing or invalid tokens on protected endpoints.
- Keep one user from editing another user profile.

API Design

Method	Endpoint	Purpose
POST	/api/auth/register	Create customer account
POST	/api/auth/login	Authenticate credentials
GET	/api/auth/me	Return current user
PUT	/api/auth/profile	Update current profile
GET	/api/auth/check-username	Check username availability
PUT	/api/auth/username	Save username

DB Design

Table	Key Fields	Use
users	id; email; username; password_hash; role	Account and access attributes

Screenshots

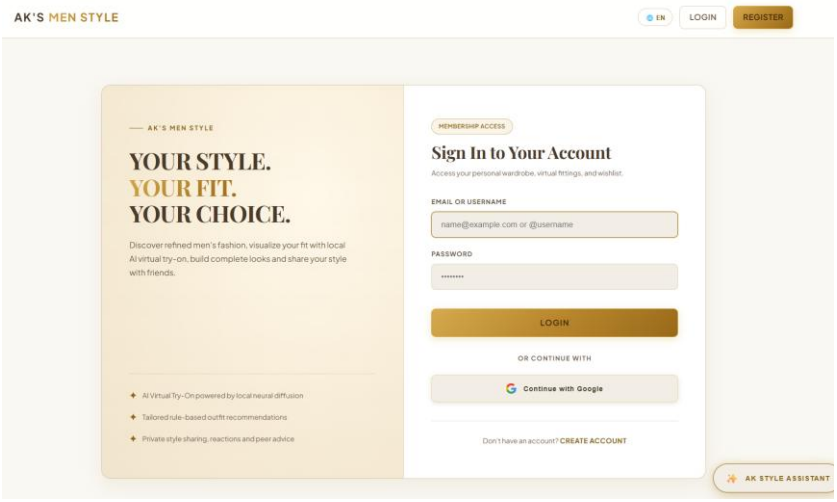


Fig 8.4 Login Page

The login page accepts account credentials and, on success, receives the JWT that opens protected pages. Google sign-in is available only when its provider credentials are configured.

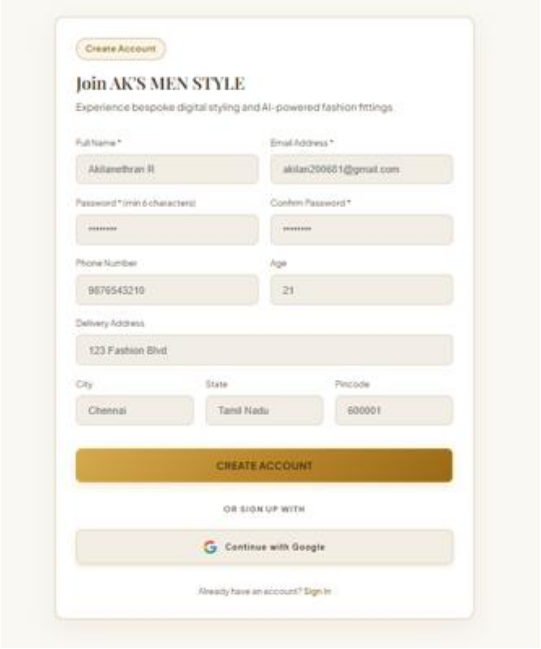


Fig 8.5 Registration Page

The registration page collects the details needed to create a customer account, such as name, email and password, and submits them to the registration API.

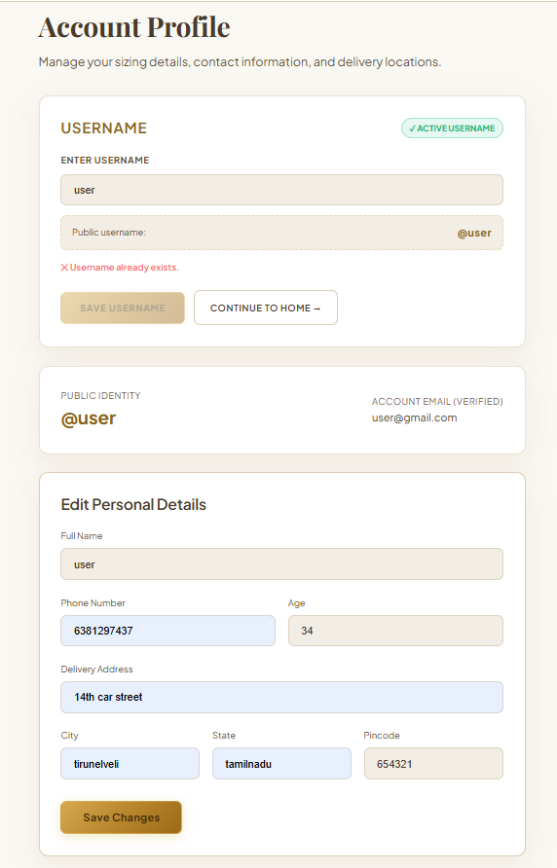


Fig 8.6 Profile and Username Page

The profile page lets the signed-in user review and update personal and delivery details, and lets a Google-created account complete a unique username.

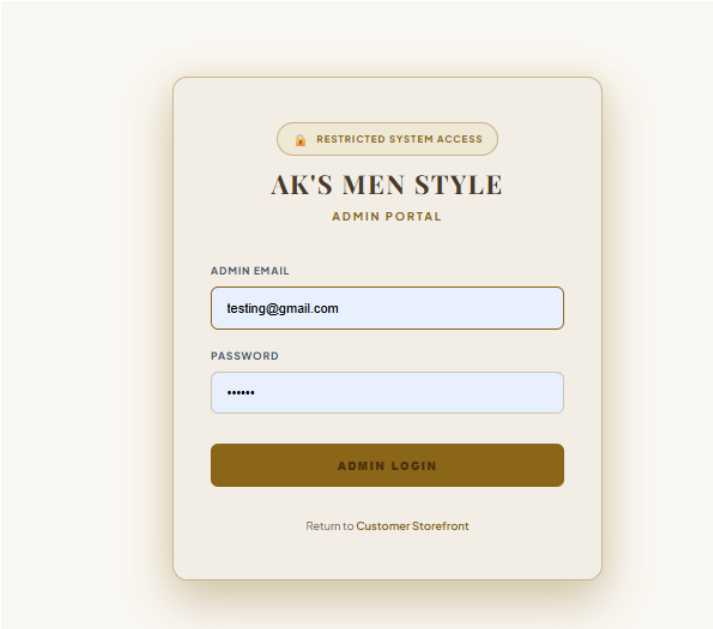


Fig 8.7 Admin Login Page

Administrators sign in through a dedicated page. Access to the management dashboard is granted only to accounts that hold the ADMIN role.

8.3 Product Catalogue and Recent Views

Purpose and Scope

This module presents products for discovery and records which items a signed-in user has viewed. Product records include catalogue text, images, prices, available stock and virtual try-on eligibility.

Functional Requirements

- List available catalogue products for browsing.
- Open a detail page for a selected product.
- Show category, price, discount, stock and garment imagery.
- Record and retrieve recently accessed items per user.
- Expose the selected product to cart, wishlist, recommendation and try-on actions.

Non Functional Requirements

- Return a clear result for an invalid product identifier.
- Keep recently accessed records isolated by user.
- Show stock and availability from persisted product data.

API Design

Method	Endpoint	Purpose
GET	/api/products	List products
GET	/api/products/:id	Get product detail
GET	/api/recently-accessed	List user recent views
POST	/api/recently-accessed	Record a product view

DB Design

Table	Key Fields	Use
products	id; name; category; price; stock; image; vton_supported	Catalogue source
recently_accessed	user_id; product_id; accessed_at	Per-user viewing history

Screenshots

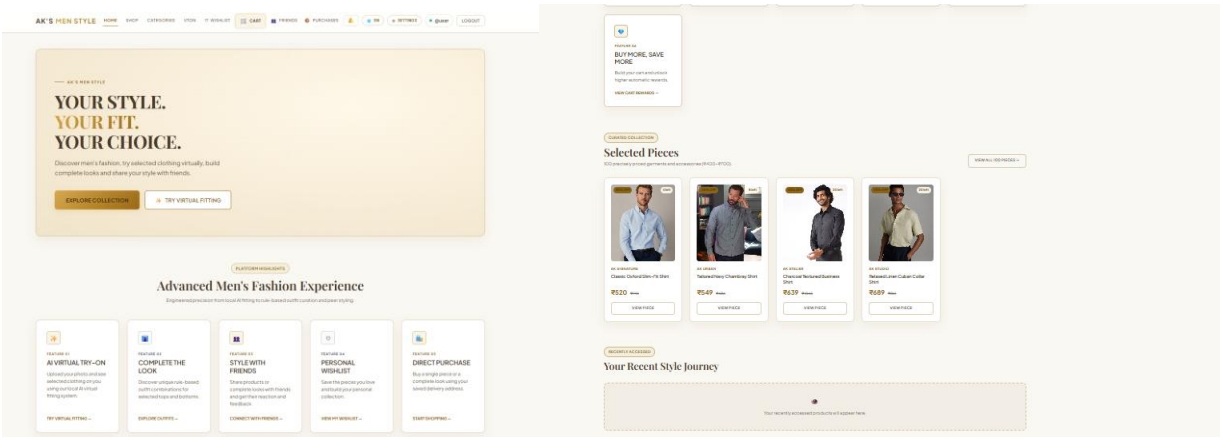


Fig 8.8 Home and Featured Products

The home page introduces the store and highlights featured products, giving shoppers an entry point into the catalogue.

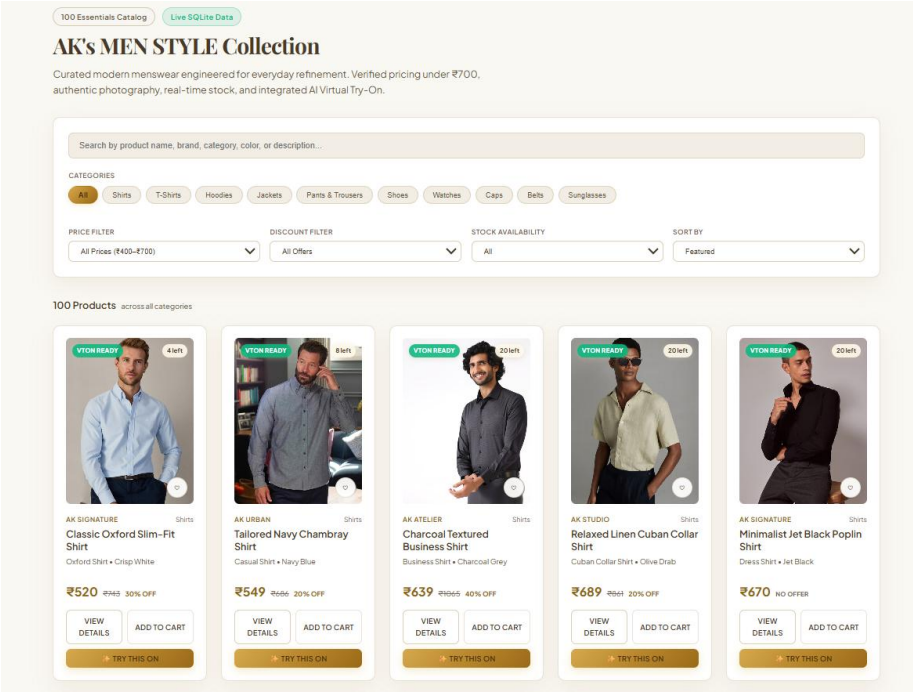


Fig 8.9 Product Catalogue

The catalogue lists available products with their image, name, price and stock information so that shoppers can browse and open a product.

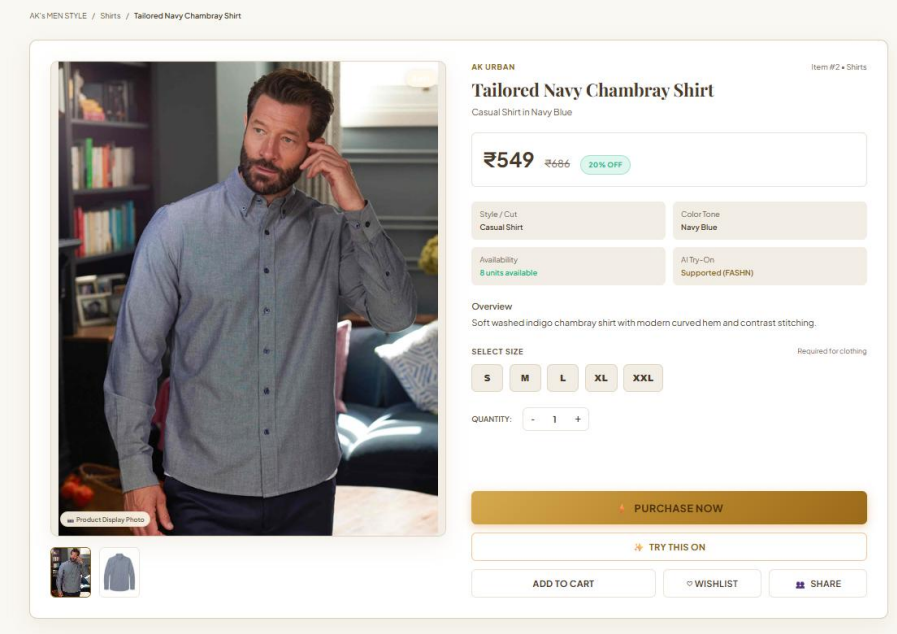


Fig 8.10 Product Detail

The detail page presents a single product with its price, sizes and stock, and offers cart, wishlist, purchase and outfit-suggestion actions. Opening it also updates the recently accessed list.

8.4 Cart and Checkout

Purpose and Scope

The cart groups selected products and size variants before purchase. Checkout calculates an order summary and applies the implemented Buy More Save More reward when the cart meets its rule conditions.

Functional Requirements

- Add a product and available size to the active cart.
- Change quantity or remove an item.
- Show subtotal and applicable cart reward.
- Submit eligible cart contents with delivery details.
- Create purchase records and decrement stock through the purchase path.

Non Functional Requirements

- Do not allow nonpositive quantities or unavailable stock to be purchased.
- Keep each cart tied to its authenticated owner.
- Perform stock and purchase updates in the same transaction where implemented.

API Design

Method	Endpoint	Purpose
GET	/api/cart	Read current cart
POST	/api/cart/items	Add a cart item
PUT	/api/cart/items/:productId	Change quantity
DELETE	/api/cart/items/:productId	Remove item
POST	/api/purchases/cart	Purchase cart contents

DB Design

Table	Key Fields	Use
cart	id; user_id	One active user cart
cart_items	cart_id; product_id; size; quantity	Selected variants
purchases	user_id; total_amount; cart_reward_amount	Completed purchase
purchase_items	purchase_id; product_id; size; quantity	Purchased units

Screenshots

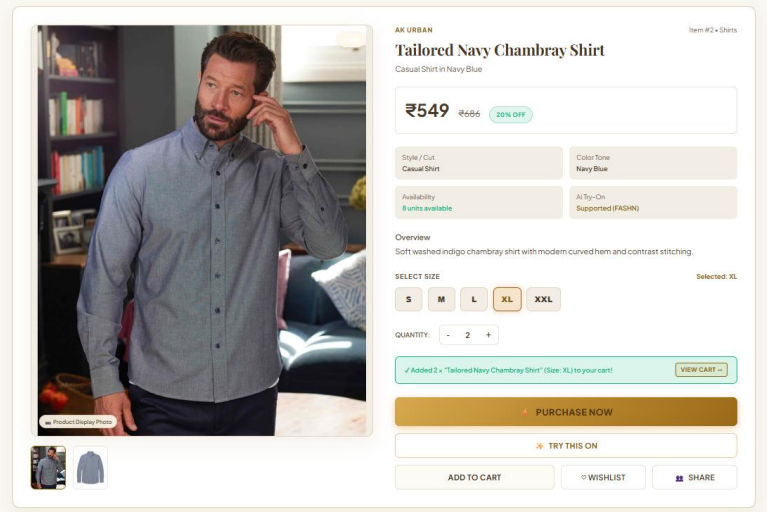


Fig 8.20 Add to Cart Action

A product is added to the cart after an available size is chosen. The cart is stored against the authenticated account.

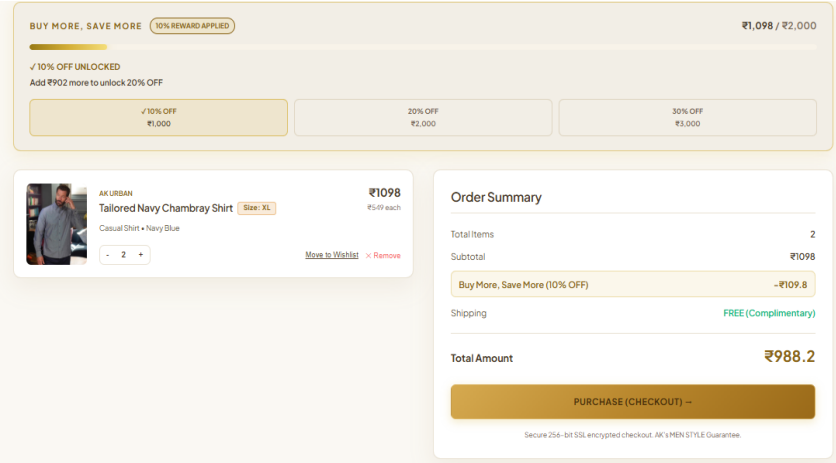


Fig 8.21 Cart Items and Quantities

The cart page lists the selected products with their size and quantity, and allows a quantity to be changed or an item to be removed.

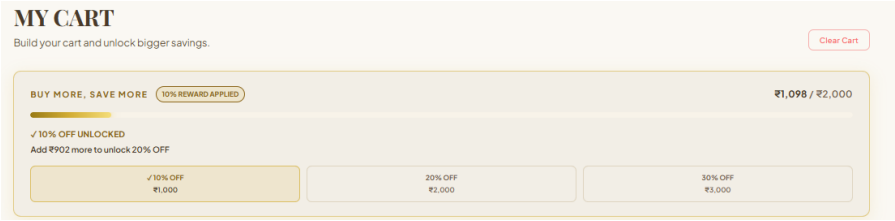


Fig 8.22 Cart Reward Summary

The summary shows the cart subtotal together with any applicable cart reward before the shopper proceeds to checkout.

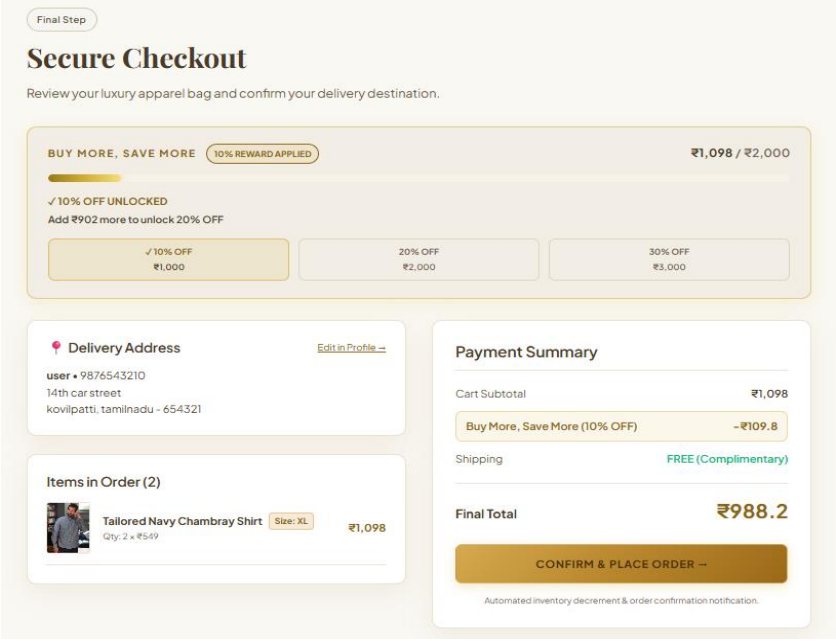


Fig 8.23 Checkout Page

The checkout page collects delivery details and shows the computed total before the cart purchase is submitted.

8.5 Purchases and Notifications

Purpose and Scope

This module stores completed purchases and their items, provides a purchase history and records in-app and configured channel communication. Single, combination and cart purchase routes share the saved purchase model.

Functional Requirements

- Purchase one product, an eligible combination or a cart.
- Save delivery data, purchased items and final amount.
- Retrieve a signed-in user purchase history and one purchase detail.
- Create and mark in-app notifications as read.
- Record email, SMS or WhatsApp delivery attempts and outcomes.

Non Functional Requirements

- Return only purchases owned by the authenticated user.
- Preserve the purchase when a nonessential external notification fails.
- Record configured, disabled or failed delivery states accurately.

API Design

Method	Endpoint	Purpose
POST	/api/purchases/product	Purchase one product
POST	/api/purchases/combo	Purchase an outfit
POST	/api/purchases/cart	Purchase cart
GET	/api/purchases	List own purchases
GET	/api/purchases/:id	Get one purchase
GET	/api/notifications	List notifications
PUT	/api/notifications/:id/read	Mark notification read

DB Design

Table	Key Fields	Use
purchases	id; user_id; purchase_type; total_amount	Purchase header
purchase_items	purchase_id; product_id; quantity; unit_price	Purchase lines
notifications	user_id; title; message; is_read	In-app messages
notification_deliveries	purchase_id; channel; status; provider_message_id	Channel delivery audit

Screenshots

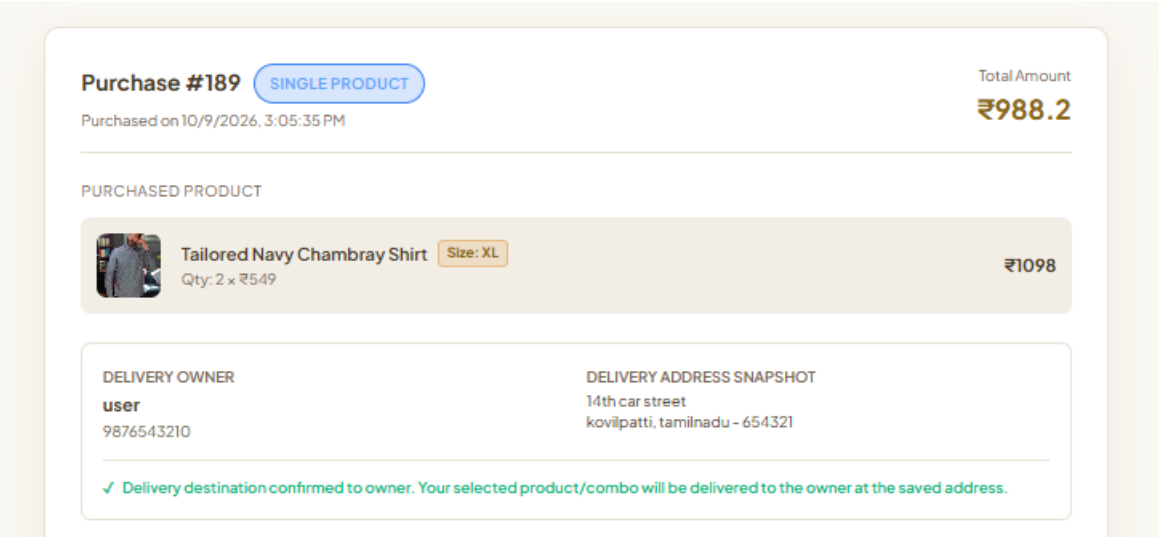


Fig 8.24 Purchase History

The purchase history page lists the customer's saved transactions with their items, delivery details and totals.

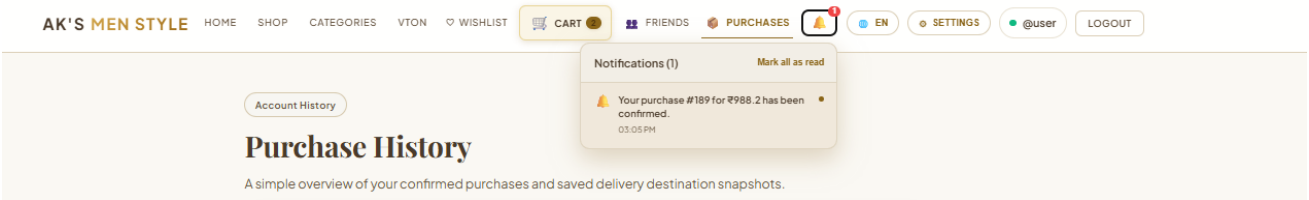


Fig 8.25 Notification Panel

The notification panel shows in-app messages created by purchases and other events, so the user can review what has happened and mark messages as read.

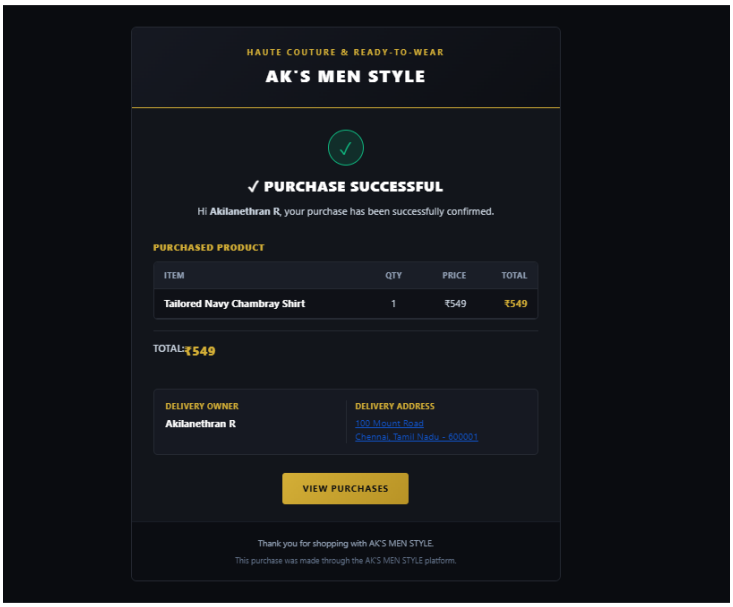


Fig 8.26 Gmail Notification

The purchase receipt delivered by email, showing the item details and delivery details of the order.

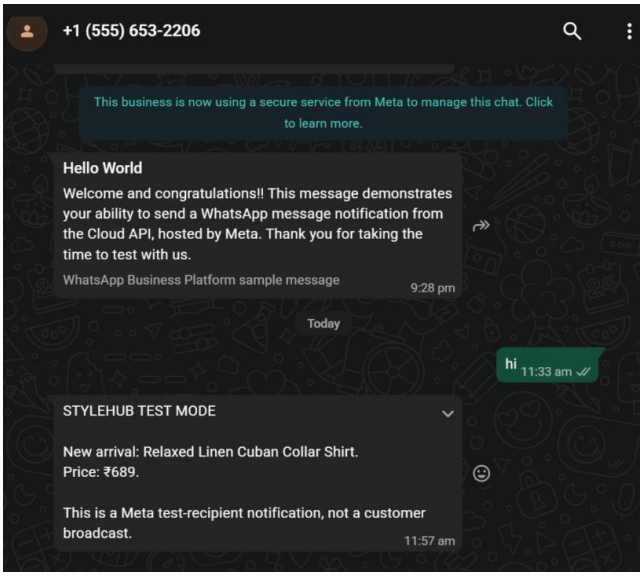


Fig 8.27 WhatsApp Notification

It shows the Meta sample message confirming that the API connection works, followed by a StyleHub message announcing a new product with its price. This confirms that StyleHub can deliver notifications to the customer's WhatsApp number.

8.6 Wishlist and Friends

Purpose and Scope

The social module stores favourite products, manages friendship requests and supports private sharing with feedback. It keeps a user wishlist separate from the cart and limits sharing to permitted relationships.

Functional Requirements

- Add and remove products from the personal wishlist.
- Find users and send, accept or reject friend requests.
- Share selected wishlist items with a friend.
- Open received shares and respond with reactions or comments.
- Show the current friend and request lists.

Non Functional Requirements

- Restrict wishlist and share access to the relevant user.
- Prevent duplicate wishlist items and invalid self-friendship records.
- Show a clear status for pending, accepted or rejected requests.

API Design

Method	Endpoint	Purpose
GET	/api/wishlist	Read wishlist
POST	/api/wishlist/items	Save product
DELETE	/api/wishlist/items/:productId	Remove product
POST	/api/friends/request	Send request
POST	/api/friends/:id/accept	Accept request
POST	/api/wishlist/shares	Create share
POST	/api/wishlist/shares/:shareId/feedback	Add feedback

DB Design

Table	Key Fields	Use
wishlists; wishlist_items	user_id; wishlist_id; product_id	Saved products
friendships	requester_id; receiver_id; status	Relationship state
wishlist_shares; wishlist_share_items	sender_id; receiver_id; share_id; product_id	Private share
wishlist_feedback	share_id; user_id; reaction; comment	Feedback

Screenshots

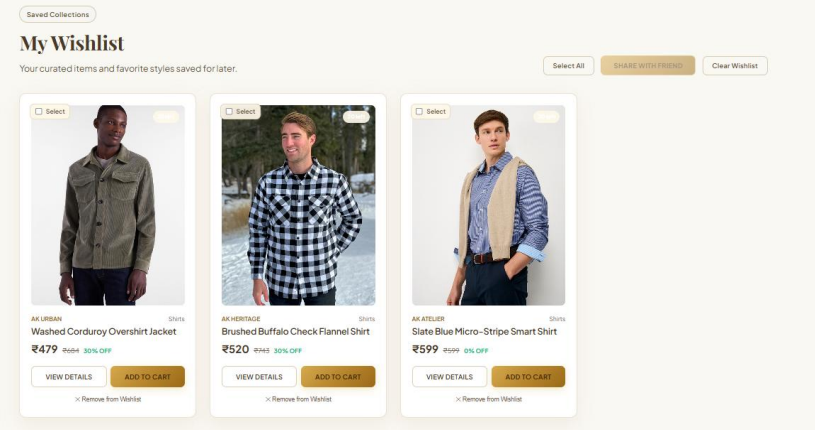


Fig 8.28 Personal Wishlist

The personal wishlist keeps saved favourites separate from the cart, so products can be revisited without a new catalogue search.

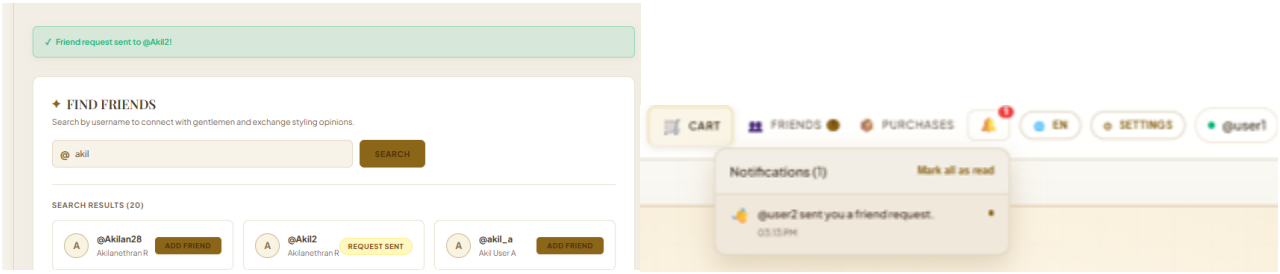


Fig 8.29 Friend Search and Requests

Users search for other members and send, accept or reject friend requests from this page.

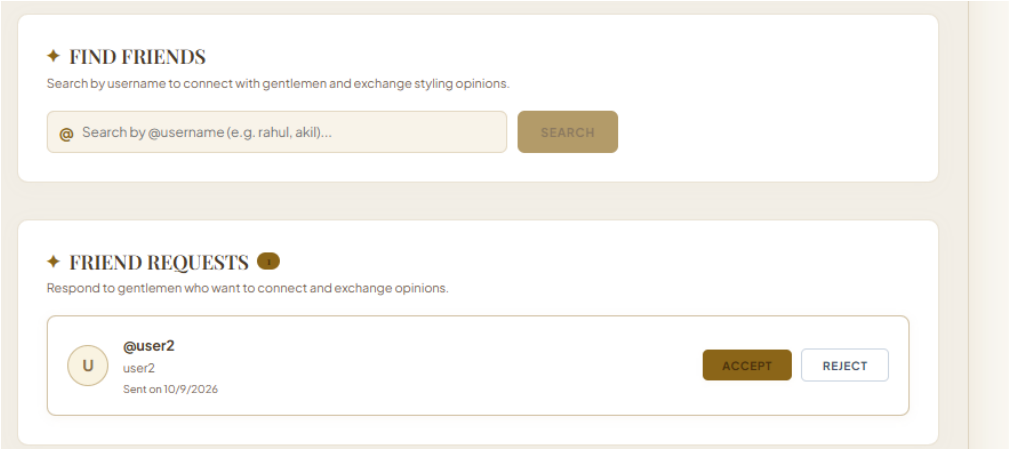


Fig 8.30 Friends List

The friends list shows accepted connections with whom selected wishlist products can be shared.

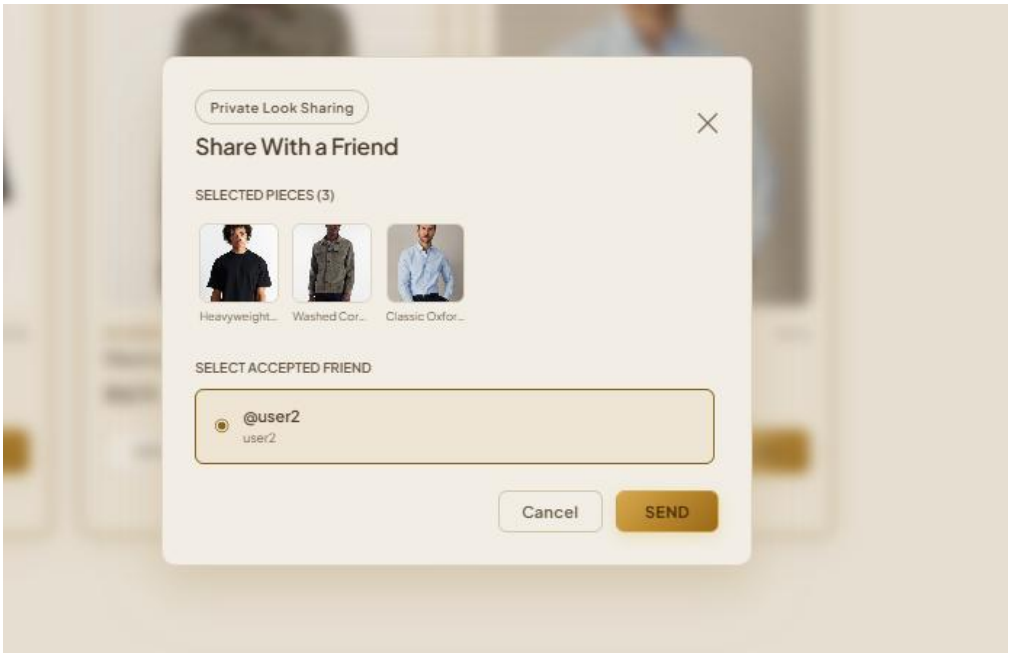


Fig 8.31 Shared Wishlist

A shared wishlist shows the products a friend has chosen to share, and lets the receiver respond to the share.

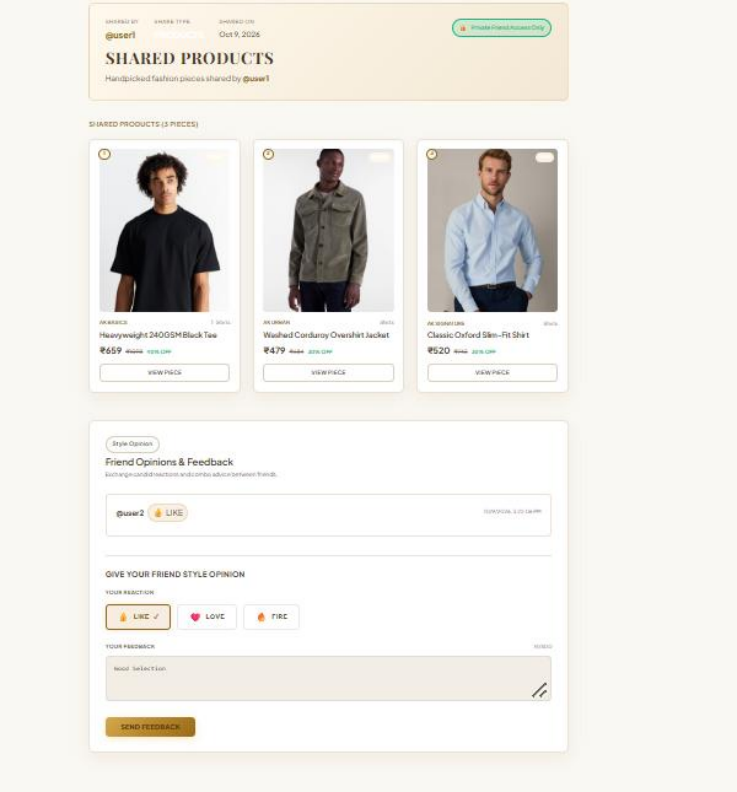


Fig 8.32 Share Feedback

` The receiver of a share can leave a reaction and a comment. Both are stored against the share and are visible for the collaborative style decision.

8.7 Recommendations and Chatbot

Purpose and Scope

` This module helps shoppers discover complementary clothing and obtain quick fashion guidance. Outfit suggestions use predefined matching rules for eligible products; the chatbot uses deterministic intent and catalogue logic.

Functional Requirements

- Return a complementary outfit for an eligible base product.
- Explain when an item is not eligible for outfit recommendations.
- Answer supported product, stock, styling and shopping questions.
- Offer links or guidance toward relevant catalogue and try-on pages.

Non Functional Requirements

- Do not present rule based results as learned predictions.

- Use current catalogue values when answering product questions.
- Return a controlled response for unsupported input or invalid product IDs.

API Design

Method	Endpoint	Purpose
GET	/api/recommendations/product/:productId	Get outfit suggestions
POST	/api/chatbot/message	Get assistant response

DB Design

Table	Key Fields	Use
products	id; category; color; price; stock	Current catalogue source
outfit_recommendations	base_product_id; recommended_product_id; slot_type	Stored recommendation links

Screenshots

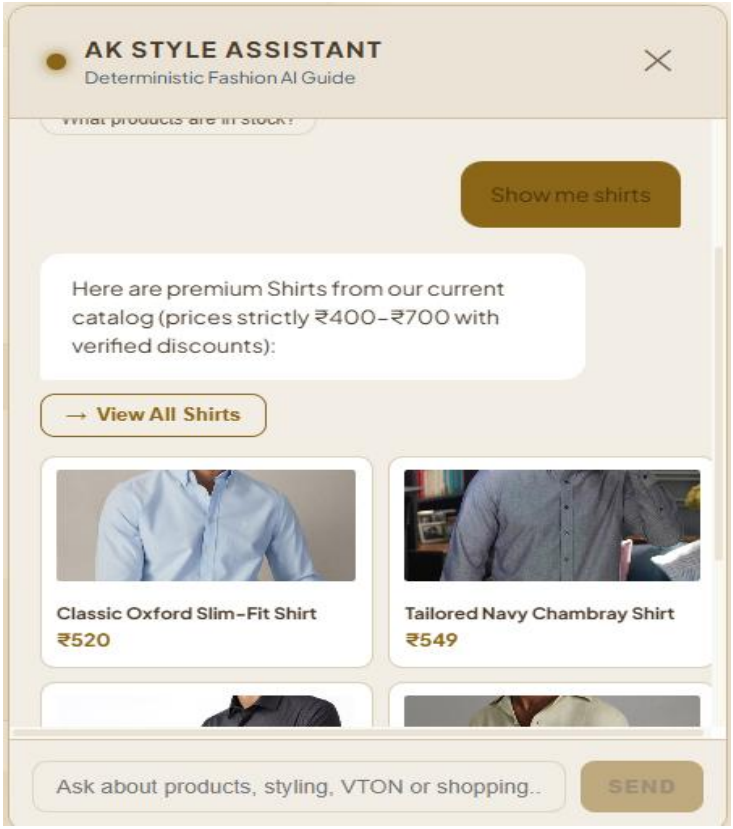


Fig 8.33 Style Assistant Chat

The style assistant answers common product and shopping questions using application rules and catalogue data, and offers suggested prompts to start a conversation.

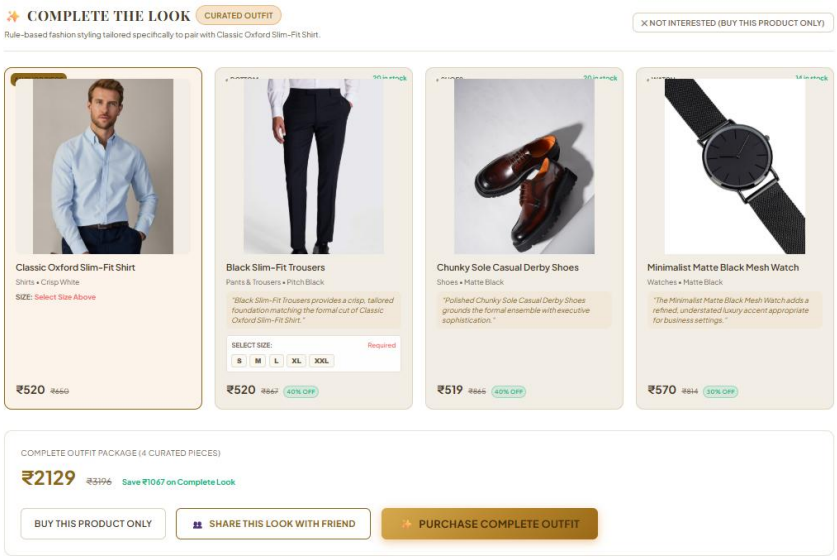


Fig 8.34 Outfit Recommendation

Opening an eligible product shows suggested matching garments with their images, names and prices.

8.8 Virtual Try-On

Purpose and Scope

Virtual try-on generates a preview of a supported garment on a user photograph. The Express API validates the request and contacts a separate local Python service that runs FASHN VTON 1.5.

The React frontend (VirtualTryOnPage.jsx and vtonService.js) guides photo upload and result viewing. The Express backend (vtonRoutes.js) validates requests and saves results in SQLite, and a Python service in vton-service runs the model. Garment-only images ensure that only the selected item changes.

Functional Requirements

- Report whether the local try-on service is ready.
- Accept a supported photo and selected product.
- Check garment eligibility and local garment image availability.
- Send the person and garment images to the Python service.
- Save and retrieve successful results for the authenticated user.

Non Functional Requirements

- Reject unsupported image types and missing uploads.
- Accept JPG, JPEG, PNG or WEBP files up to 15 MB; protect results by owner.
- Report service unavailability without blocking ordinary shopping.

Implementation Details

Frontend Implementation

- Page: VirtualTryOnPage.jsx in the frontend pages folder opens from the Virtual Try-On navigation link or from the Try This On action on a product.
- Service calls: vtonService.js in the frontend services folder sends the status, try-on and history requests through the shared API service with the JWT.
- Readiness: the page checks /api/vton/status when it loads and shows the FASHN VTON v1.5 engine badge and the CUDA Online indicator. If the service is not ready, the page reports it and ordinary shopping continues.

- Guided steps: a progress strip shows Your Photo, Selected Garment, Generate Virtual Try-On and AI Generated Result.
- Upload panel: Step 1 accepts JPG, JPEG, PNG or WEBP photographs up to 15 MB, shows a preview with a Change Photo button and keeps Generate disabled until a photograph is ready.
- Garment panel: Step 2 shows the garment-only image with brand, name, category, type, colour, price and stock status.
- Result panel: Step 3 shows the generated image with Download Result, Try Another Photo and Try Another Product actions.

Backend Implementation

- Routes: vtonRoutes.js exposes the four /api/vton endpoints and identifies the user from the JWT on protected requests.
- Upload handling: Multer receives the multipart request and rejects missing files, unsupported image types and files above the size limit.
- Validation: the handler confirms that the product exists, that vton_supported is set, that a vton_garment_category is present and that the local garment image is available.
- Inference: Express sends the person and garment images to the Flask service in the backend vton-service directory, which runs FASHN VTON 1.5 and returns the output image.
- Persistence: the output path is saved in try_on_results with user_id, product_id, user_image_path, garment_image_path, generated_image_path, category, status and created_at.
- Error handling: an unavailable service, an unsupported garment or a failed generation returns an error response without affecting the cart, purchase or other modules.

API Design

Method	Endpoint	Purpose
GET	/api/vton/status	Check service readiness
POST	/api/vton/try-on	Generate try-on result
GET	/api/vton/history	List own results
GET	/api/vton/results/:id	Get one owned result

DB Design

Table	Key Fields	Use
products	id; garment_image; vton_supported; vton_garment_category	Eligible garment metadata
try_on_results	user_id; product_id; generated_image_path; status	Generated result record

Screenshots

The screenshots follow one complete try-on for each garment type. In every case the user uploads a clear full-body photograph, the selected product appears as a garment-only image, and the generated preview applies only the chosen clothing item to that photograph.

Garment	Brand	Category	Result
Classic Oxford Slim-Fit Shirt	AK Signature	Shirts	Fig 8.37
Denim Shirt	US Polo Assn.	Shirts	Fig 8.39
Black Slim-Fit Trousers	AK Signature	Pants & Trousers	Fig 8.41

Shirt Try-On

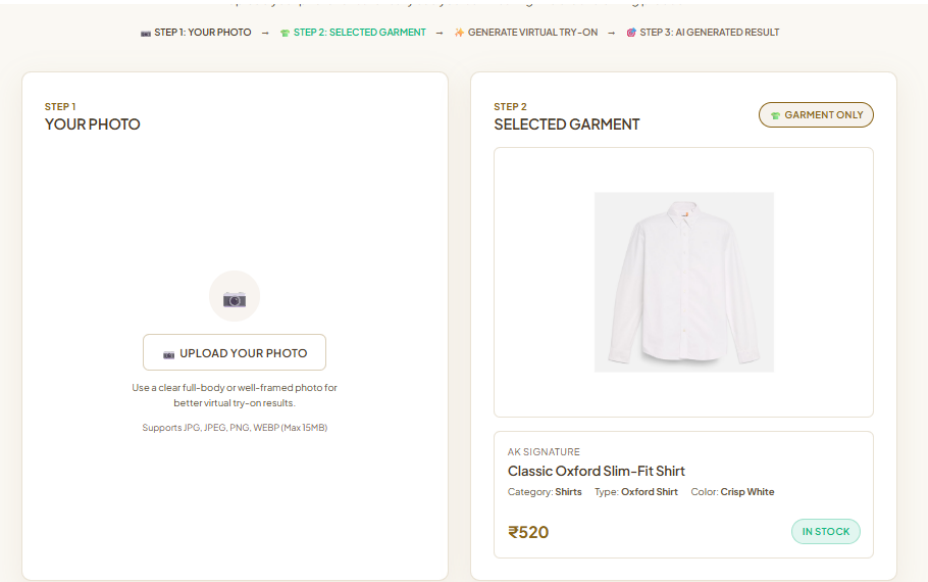


Fig 8.35 Virtual Try-On Entry

The fitting room before any photograph is chosen. The header shows the FASHN VTON v1.5 engine badge and the progress strip marks Step 1 as active. The upload panel accepts JPG, JPEG, PNG or WEBP files up to 15 MB, and the Generate Virtual Try-On button stays disabled until a photograph is added

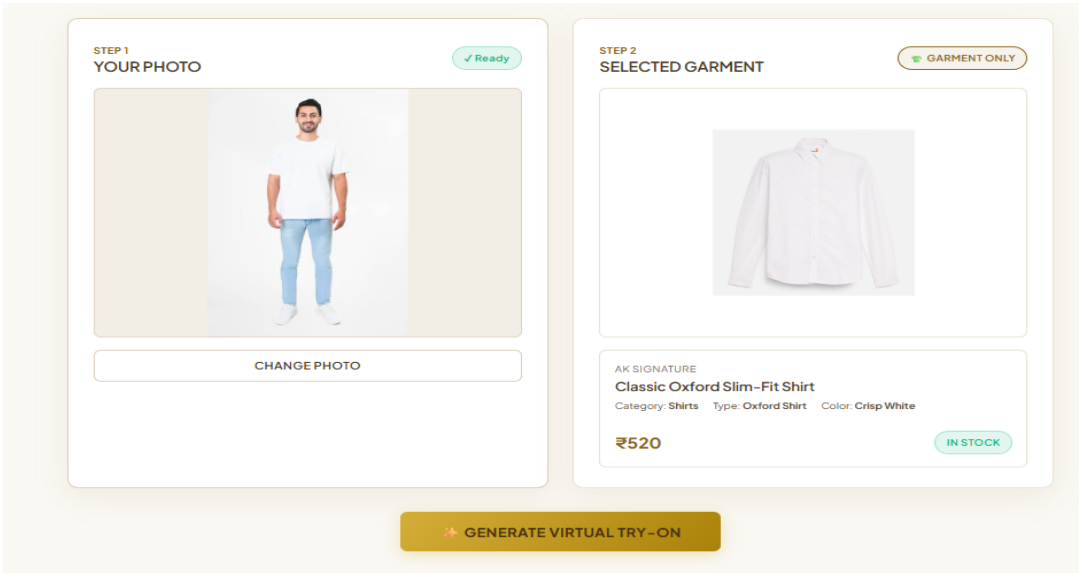


Fig 8.36 Photo and Garment Selection

After the upload, Step 1 shows the Ready status and a Change Photo button. Step 2 shows the garment-only image of the selected shirt with its brand, category, type, colour, price and stock status.

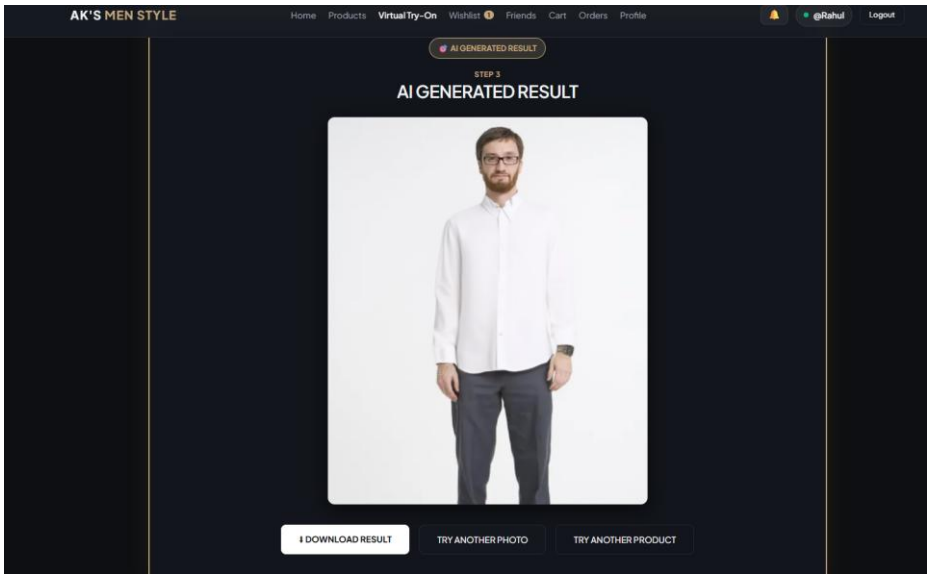


Fig 8.37 Generated Try-On Result

Step 3 shows the generated preview with the selected shirt applied to the uploaded photograph. Download Result, Try Another Photo and Try Another Product let the user save the image or repeat the try-on.

Additional Shirt Example

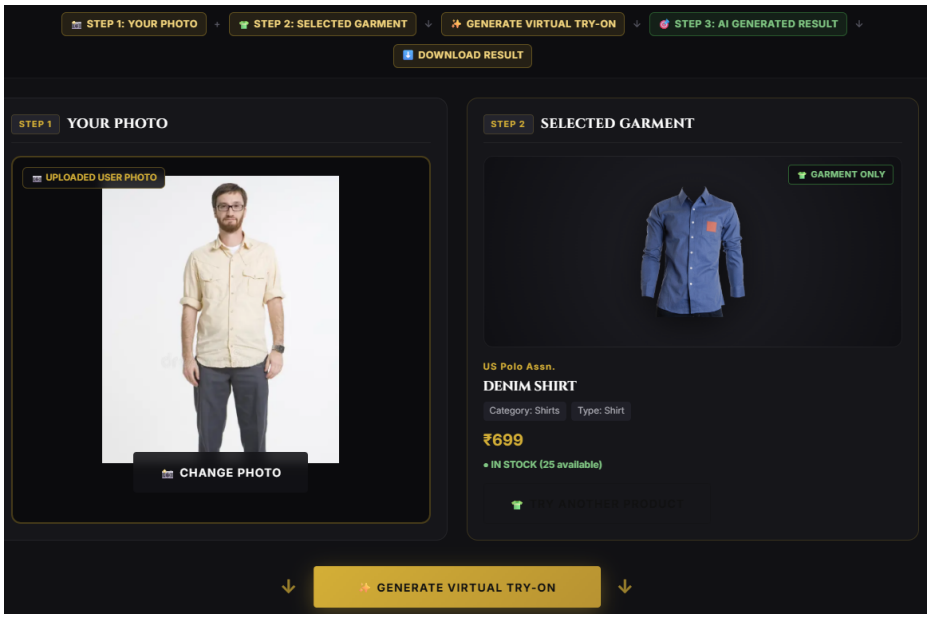


Fig 8.38 Denim Shirt Selected for Try-On

A second shirt example. Step 1 shows the uploaded photograph with its Uploaded User Photo label, and Step 2 shows the US Polo Assn. Denim Shirt with its category, type, price and available stock. The Generate Virtual Try-On button is enabled.

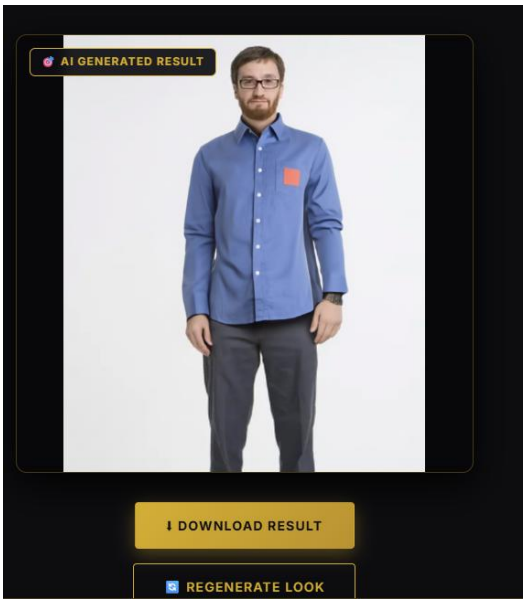


Fig 8.39 Denim Shirt Try-On Result

The generated preview shows the denim shirt on the same photograph. Download Result and Regenerate Look buttons appear below the image so the user can save the result or request another generation.

Trousers Try-On

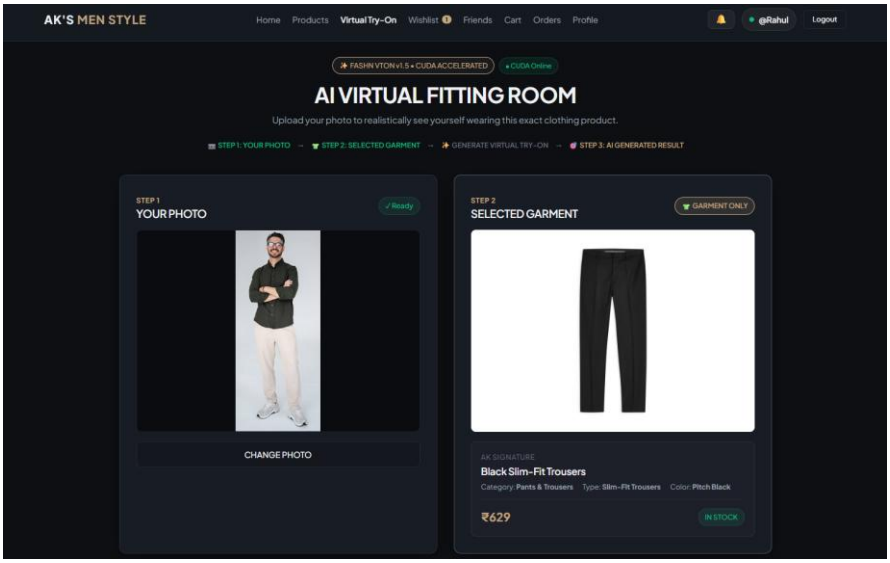


Fig 8.40 Trousers Selected for Try-On

A lower-body example. The Black Slim-Fit Trousers (AK Signature, Pants & Trousers, Pitch Black) appear as a garment-only image beside the uploaded full-body photograph, and the header badges show that the try-on engine is online.

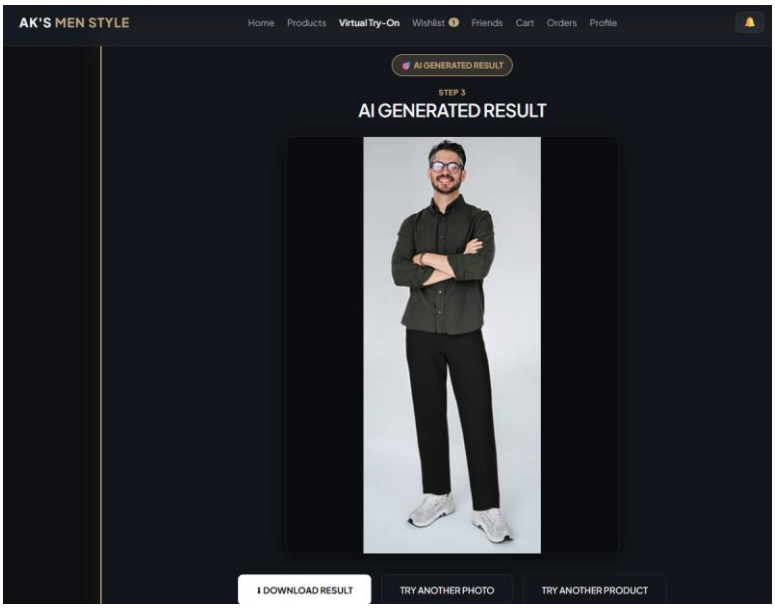


Fig 8.41 Trousers Try-On Result

The generated preview shows the black trousers worn in place of the original pants while the shirt and background from the uploaded photograph are kept. Download Result, Try Another Photo and Try Another Product are available below the result.

8.9 Admin Dashboard

Purpose and Scope

The administrator module provides an account restricted view of catalogue and application activity. It offers summary metrics, users, purchases, virtual try-on activity and limited product updates.

Functional Requirements

- Require an ADMIN role before exposing routes.
- Display metrics and recent activity.
- List products, users and purchases for administration.
- Update supported product fields through the admin route.
- Show virtual try-on and email status information.

Non Functional Requirements

- Enforce the role on the backend even if a page route is hidden.
- Avoid exposing password hashes in user summaries.
- Validate editable product fields before writing them.

API Design

Method	Endpoint	Purpose
GET	/api/admin/metrics	Read dashboard metrics
GET	/api/admin/products	List products
PATCH	/api/admin/products/:id	Update product
GET	/api/admin/users	List users
GET	/api/admin/purchases	List purchases
GET	/api/admin/vton-stats	Read try-on metrics

DB Design

Table	Key Fields	Use
users	id; role; email	Role and user summary
products	id; price; stock; status	Catalogue editing
purchases	id; user_id; total_amount	Purchase summary
try_on_results	user_id; product_id; status	Try-on activity

Screenshots

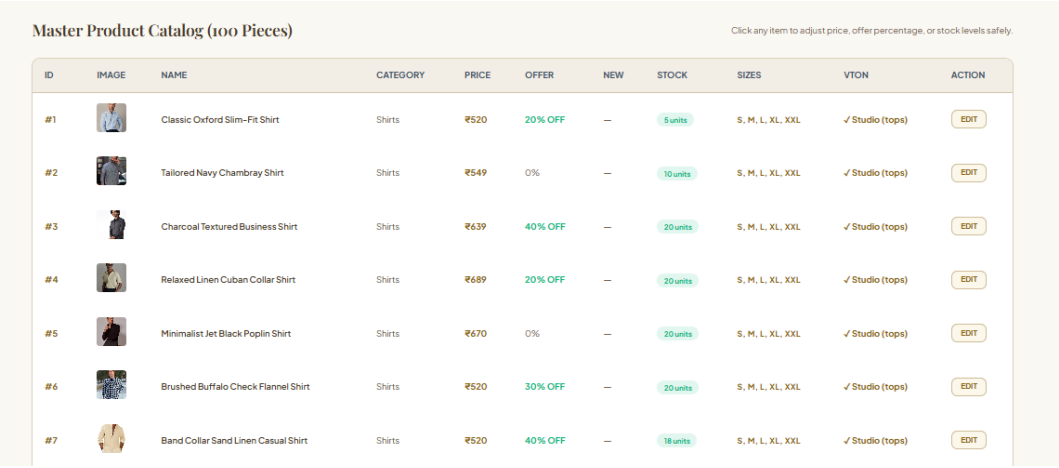


Fig 8.42 Admin Product Management

The product management view lets the administrator review the catalogue and update product records such as price and stock.

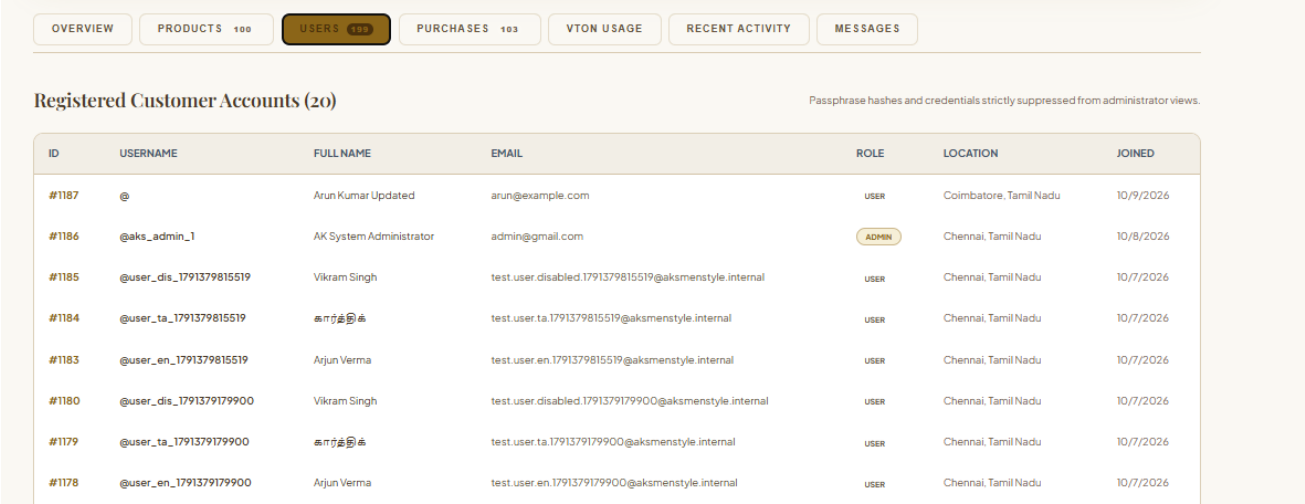


Fig 8.43 Admin Users and Purchases

These views list registered customer accounts and purchase records so that the administrator can review them.

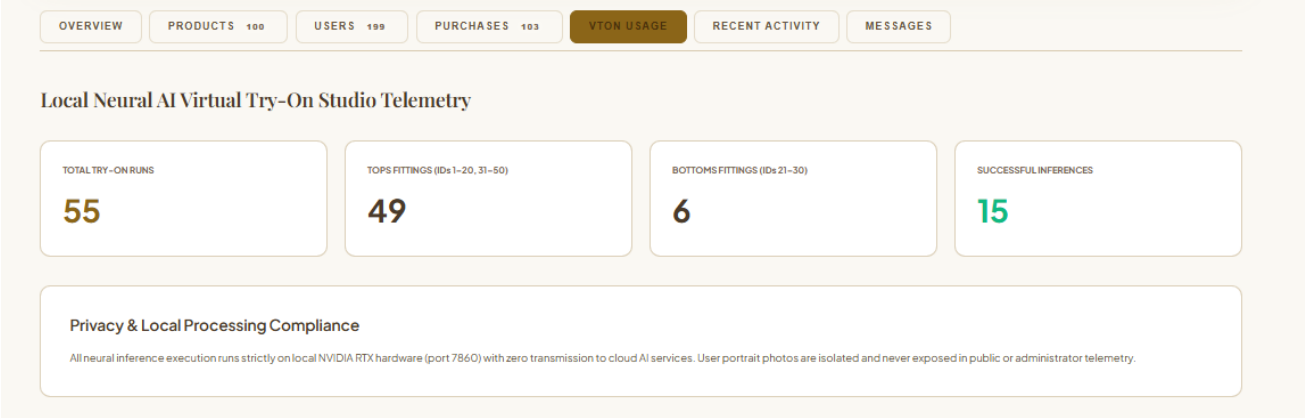


Fig 8.44 Admin Activity and VTON Metrics

The metrics panel summarises recent virtual try-on activity, including attempt counts and their results, for administrative monitoring.

8.10 Theme and Language Preferences

Purpose and Scope

This module lets a user select the application appearance, interface language and communication preferences. English, Tamil and Hindi translations are provided by the current frontend language context.

Functional Requirements

- Switch between available interface languages.
- Remember the selected language across page reloads.

- Change among supported appearance themes.
- Save email, SMS and WhatsApp notification preferences.
- Retrieve the signed-in user settings and delivery history.

Non Functional Requirements

- Use a supported language when a saved value is unavailable.
- Keep account preferences associated with the correct user.
- Avoid claiming external delivery from a preference switch alone.

API Design

Method	Endpoint	Purpose
GET	/api/settings	Read user settings
PUT	/api/settings	Save preferences
GET	/api/settings/notifications/history	Read delivery history

DB Design

Table	Key Fields	Use
users	id; theme_preference; language_preference; email_notifications; sms_notifications; whatsapp_notifications	Saved user choices
notification_deliveries	user_id; channel; status	Communication history

Screenshots

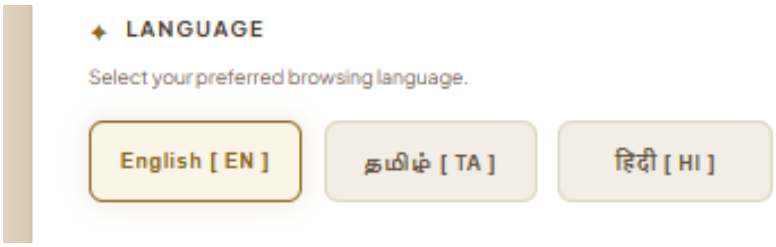


Fig 8.46 Language Selector

The language selector switches the interface between English, Tamil and Hindi, and the choice is remembered when the page is reloaded.

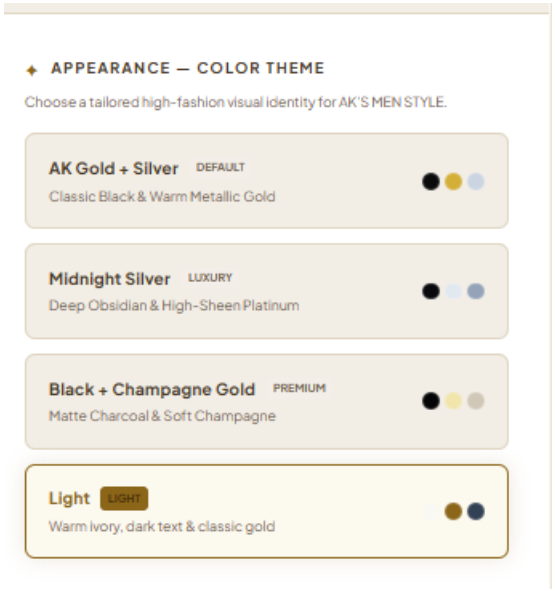


Fig 8.47 Theme Selector

The theme selector changes the appearance of the application between the supported themes and keeps the selection.

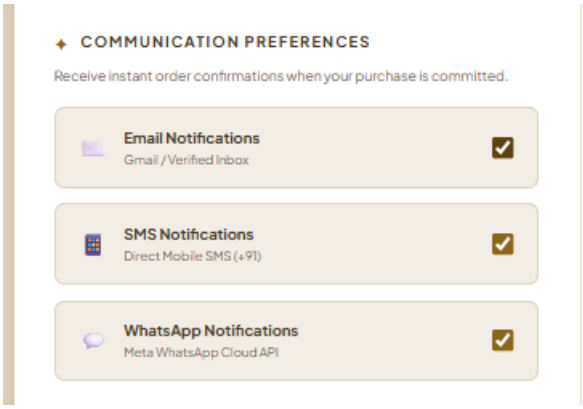


Fig 8.48 Communication Preferences

Email, SMS and WhatsApp notification preferences are switched on or off here and saved to the user account.

CHAPTER 9

CONCLUSION

StyleHub brings the main operations of a men fashion shopping application into one system: protected accounts, product browsing, cart and purchase handling, wishlists, friend sharing, rule based suggestions, administration, and saved preferences. The SQLite schema and REST routes provide a coherent basis for these modules.

Virtual try-on extends the project through a local third party model, while Google sign-in and communication channels depend on external configuration. The report diagrams and screenshot spaces support later submission updates without changing the underlying technical description.

The nine modules documented in this report work as one system rather than as separate pages. Authentication ties every cart, wishlist, purchase, notification and try-on record to a single account, while the shared product catalogue links browsing, recommendations, the chatbot and checkout. A shopper can therefore move from discovering a garment to saving it, sharing it with a friend, trying it on virtually and buying it without leaving the application.

The design choices also support reliability and security. The layered structure of React pages, Express routes, controllers and services keeps responsibilities separate and makes each module easier to test. Passwords are stored as hashes, protected endpoints require a valid JWT, administrator routes are checked again on the backend, and the authenticated user ID is used to keep purchases and social records private. SQLite foreign keys and transactional stock updates keep catalogue, cart and purchase data consistent, and optional services such as virtual try-on and external messaging report their availability separately so that the core shopping flow keeps working when they are unavailable.

The current implementation has limits that should be stated plainly. Outfit suggestions and the chatbot follow defined rules and catalogue data and do not learn from user behaviour. Virtual try-on depends on a local Python service, the FASHN VTON model weights and suitable hardware. Email, SMS and WhatsApp delivery depend on provider credentials being configured. SQLite is well suited to a single-server project but is not intended for heavy concurrent use, and payment gateway integration is outside the scope documented here.

Several enhancements would extend the project naturally: integrating an online payment gateway and order tracking; moving the database to a server-based engine for larger deployments; replacing the rule-based recommender with one trained on browsing and purchase history; hosting the

try-on service on dedicated GPU infrastructure; broadening the automated test suite; and adding further interface languages. The existing module boundaries and REST interfaces allow these changes to be made without redesigning the application.

In summary, StyleHub meets the objectives set out at the start of the report. It delivers a working men's fashion e-commerce platform that combines secure shopping, social styling, rule-based assistance and AI-assisted virtual try-on, and it provides a documented foundation for further development.